

Guida di Sviluppo

SPM — Modulo 2

*Partitioned Hash Join with Duplicates:
From Partition Mapping to Full Join*

Obiettivo: Implementare una versione parallela (C++ threads) del Partitioned Hash Join con duplicati, partendo dal codice sequenziale fornito (`hashjoin_seq.cpp`), e produrre un report con analisi delle performance.

Deadline: 20 Aprile (soft deadline)

Deliverables: codice sorgente + Makefile + README + report PDF (max 5 pagine)

Corso: Sistemi Paralleli e Multi-core (SPM)
A.Y. 2025–2026 — Prof. Massimo Torquati

Indice

1	Panoramica dell’Algoritmo	2
1.1	Il problema: cos’è un Join e perché è costoso?	2
1.2	L’idea del Partitioned Hash Join: “dividi e conquista”	2
1.3	Collegamento con il Modulo 1: la funzione di mapping	2
1.4	Le fasi dell’algoritmo: visione d’insieme	3
1.5	Esempio numerico completo (passo per passo)	3
1.5.1	Step 1 — Mapping ($\text{key} \rightarrow \text{partition_id}$)	3
1.5.2	Step 2 — Histogram	4
1.5.3	Step 3 — Prefix Sum (offset di inizio)	4
1.5.4	Step 4 — Scatter (riordino per partizione contigua)	4
1.5.5	Step 5 — Build (hash table locale su ogni R_p)	4
1.5.6	Step 6 — Probe (cercare match da S_p)	5
1.5.7	Step 7 — Accumulazione	5
1.6	Perché partizionare è vantaggioso	5
1.7	La gestione dei duplicati: perché “with Duplicates”	5
1.8	Checksum: verifica senza materializzare le coppie	6
1.9	Mappa concettuale: Modulo 1 $\rightarrow$ Modulo 2	6
2	Teoria: Concetti Chiave dalle Lezioni	7
2.1	Metriche di Performance (Lezione 10)	7
2.2	Strong vs Weak Scaling (Lezione 10)	7
2.3	Work-Span Model (Lezione 15)	7
2.4	Tipi di Parallelismo (Lezione 11)	7
2.5	Data Partitioning e Workload Balancing (Lezione 14)	7
2.6	C++ Threads (Lezione 13)	8
2.7	Cache e False Sharing (Lezione 5–6)	8
3	Step 0: Setup dell’Ambiente	9
3.1	Struttura del progetto	9
3.2	Makefile	9
3.3	Compilazione e test iniziale	9
4	Step 1: Integrare la Funzione di Mapping del Modulo 1	9
5	Step 2: Capire il Codice Sequenziale	10
5.1	Generazione dei dati	10
5.2	Histogram — $O(N)$	10
5.3	Prefix Sum — $O(P)$	10
5.4	Scatter — $O(N)$	10
5.5	Join locale (Build + Probe)	10
6	Step 3: Analisi delle Fasi e Opportunità di Parallelismo	10
7	Step 4: Implementazione Parallela	11
7.1	Architettura generale	11
7.2	Histogram parallelo	11
7.3	Scatter parallelo lock-free (raccomandato)	11
7.4	Join locale parallelo	12
7.5	Alternativa: distribuzione dinamica	12
7.6	Thread Pool vs Spawn-Join	12
8	Step 5: Strategia di Validazione (Correctness)	13

9	Step 6: Performance Evaluation	13
9.1	Strong Scaling	13
9.2	Weak Scaling	13
9.3	Breakdown per fase	13
9.4	Regole di benchmarking	13
10	Step 7: Scrivere il Report	14
11	Checklist Finale	14
12	Appendice: Comandi Utili & Riferimenti	14
12.1	Compilazione	14
12.2	Profilazione	15
12.3	Riferimenti alle lezioni	15

1 Panoramica dell'Algoritmo

1.1 Il problema: cos'è un Join e perché è costoso?

Immagina di avere due tabelle di un database:

Relazione R (es. “Ordini”)

key
3
7
3
5
2

Relazione S (es. “Prodotti”)

key
5
3
3
7
1
3
2
5

L'operazione di **join** trova tutte le coppie (r, s) dove $r.key = s.key$. In SQL: `SELECT * FROM R JOIN S ON R.key = S.key`.

Approccio naive Confronta ogni record di R con ogni record di S:

Complessità: $O(N_R \times N_S)$

Con 10 milioni di record per tabella $\Rightarrow 10^{14}$ confronti. **Inaccettabile**.

Hash join classico Costruisci una hash table su R, poi proli con S:

1. Costruisci `hash_table` da R

$O(N_R)$

2. Per ogni $s \in S$: cerca $s.key$ nella hash table

$O(N_S)$ in media

Complessità totale: $O(N_R + N_S)$. Ma se R ed S sono enormi, la hash table non entra in cache e ogni lookup produce *cache miss* $\Rightarrow$ lento nella pratica.

1.2 L'idea del Partitioned Hash Join: “dividi e conquista”

Idea Centrale

Se applico la **stessa** funzione hash $h(key)$ sia a R che a S, allora tutti i record con la stessa chiave finiranno nella **stessa partizione**.

Questo significa che posso fare il join **separatamente** su ogni partizione, perché un record in $R_{partizione_0}$ non potrà *mai* matchare un record in $S_{partizione_1}$ (hanno hash diverse!).

Concetto visivo:

PRIMA (dati mescolati):

R = [3, 7, 3, 5, 2]
S = [5, 3, 3, 7, 1, 3, 2, 5]

DOPO (partizionati con $P = 3$):

R_0 = [2] S_0 = [1, 2]
R_1 = [3, 3, 5] S_1 = [5, 3, 3, 3, 5]
R_2 = [7] S_2 = [7]

Ogni coppia (R_p, S_p) è **indipendente**!

1.3 Collegamento con il Modulo 1: la funzione di mapping

Nel Modulo 1 hai implementato e ottimizzato una funzione:

$$h(key) \rightarrow partition_id \in \{0, 1, \dots, P-1\}$$

La tua implementazione usa **XOR-fold + Fibonacci multiply-shift a 32 bit**:

```

1 static constexpr uint32_t HASH_A32 = 0x9E3779B9u; // floor(2^32 / phi)
2
3 inline uint32_t compute_partition_id(uint64_t key, unsigned shift) {
4     uint32_t k_lo = static_cast<uint32_t>(key); // meta' bassa
5     uint32_t k_hi = static_cast<uint32_t>(key >> 32); // meta' alta
6     uint32_t mixed = k_lo ^ k_hi; // XOR-fold
7     return (mixed * HASH_A32) >> shift; // shift = 32 - log2(P)
8 }

```

Listing 1: Hash function dal Modulo 1

Connessione Modulo 1 → Modulo 2

Nel Modulo 1 hai costruito e benchmarkato questa funzione *da sola* (throughput in Mkeys/s, distribuzione tra partizioni). Nel Modulo 2, questa funzione diventa il “mattoncino” usato **dentro** un algoritmo più grande. Viene chiamata $2 \times (N_R + N_S)$ volte (una volta nell’histogram e una nello scatter, per ciascuna relazione).

Il codice fornito dal professore usa una hash semplicistica ($\text{key} \& (P-1)$). Tu devi **sostituirla** con la tua.

1.4 Le fasi dell’algoritmo: visione d’insieme

Partitioned Hash Join — Pipeline

Macro-Fase A: Partizionamento (applicata sia a R che a S)

1. Histogram (conteggi) → 2. Prefix Sum (offset) → 3. Scatter (riordino)

Macro-Fase B: Join locale (per ogni partizione $p = 0 \dots P-1$, indipendente)

4. Build (hash table su R_p) → 5. Probe (cerca match da S_p)

Macro-Fase C: Accumulazione — Somma `join_count` e checksum di tutte le partizioni

1.5 Esempio numerico completo (passo per passo)

Usiamo dati piccoli per tracciare l’intero algoritmo a mano.

Parametri dell’esempio

$P = 4$ partizioni, $h(\text{key}) = \text{key} \bmod 4$ (semplificata per leggibilità)

R	indice: 0	1	2	3	4	5	6	7	S	indice: 0	1	2	3	4	5
chiave	5	2	8	3	2	5	7	4	chiave	3	5	2	8	5	7

1.5.1 Step 1 — Mapping ($\text{key} \rightarrow \text{partition_id}$)

Applichiamo $h(\text{key}) = \text{key} \bmod 4$:

R chiave	5	2	8	3	2	5	7	4
partition_id	1	2	0	3	2	1	3	0
	5%4	2%4	8%4	3%4	2%4	5%4	7%4	4%4
S chiave			3	5	2	8	5	7
partition_id			3	1	2	0	1	3

Il mapping **non sposta nulla**: associa solo un `partition_id` ad ogni record.

1.5.2 Step 2 — Histogram

Scorriamo i `partition_id` e contiamo quanti record per partizione:

	part 0	part 1	part 2	part 3
hist_R	2 (key 8,4)	2 (key 5,5)	2 (key 2,2)	2 (key 3,7)
hist_S	1 (key 8)	2 (key 5,5)	1 (key 2)	2 (key 3,7)

A cosa serve? Sapere quanti record ha ogni partizione permette di pre-allocare lo spazio nell'array di output e calcolare dove inizia ciascuna partizione.

1.5.3 Step 3 — Prefix Sum (offset di inizio)

L'exclusive prefix sum converte i conteggi in posizioni di inizio:

	part 0	part 1	part 2	part 3	formula
hist_R	2	2	2	2	
begin_R	0	2	4	6	$\text{begin}[i] = \sum_{j < i} \text{hist}[j]$
end_R	2	4	6	8	$\text{end}[i] = \text{begin}[i] + \text{hist}[i]$

Mappa delle posizioni nell'array output di R:

part 0		part 1		part 2		part 3	
pos 0	pos 1	pos 2	pos 3	pos 4	pos 5	pos 6	pos 7

Analogamente per S: `begin_S` = [0, 1, 3, 4], `end_S` = [1, 3, 4, 6].

1.5.4 Step 4 — Scatter (riordino per partizione contigua)

Scorriamo R originale e copiamo ogni record nella posizione corretta, usando un **cursore** per partizione:

Record	Azione
R[0] key=5, pid=1	<code>out[next[1]=2] ← 5, next[1]++ → 3</code>
R[1] key=2, pid=2	<code>out[next[2]=4] ← 2, next[2]++ → 5</code>
R[2] key=8, pid=0	<code>out[next[0]=0] ← 8, next[0]++ → 1</code>
R[3] key=3, pid=3	<code>out[next[3]=6] ← 3, next[3]++ → 7</code>
R[4] key=2, pid=2	<code>out[next[2]=5] ← 2, next[2]++ → 6</code>
R[5] key=5, pid=1	<code>out[next[1]=3] ← 5, next[1]++ → 4</code>
R[6] key=7, pid=3	<code>out[next[3]=7] ← 7, next[3]++ → 8</code>
R[7] key=4, pid=0	<code>out[next[0]=1] ← 4, next[0]++ → 2</code>

Array R riordinato:

part 0		part 1		part 2		part 3	
8	4	5	5	2	2	3	7

Analogamente, **S riordinato**:

8	5	5	2	3	7
---	---	---	---	---	---

1.5.5 Step 5 — Build (hash table locale su ogni R_p)

Per ogni partizione scansioniamo R_p e contiamo le occorrenze di ogni chiave:

Partizione	R_p	countR
$p = 0$	[8, 4]	{8:1, 4:1}
$p = 1$	[5, 5]	{5: 2 } ← <i>chiave 5 duplicata!</i>
$p = 2$	[2, 2]	{2: 2 } ← <i>chiave 2 duplicata!</i>
$p = 3$	[3, 7]	{3:1, 7:1}

1.5.6 Step 6 — Probe (cercare match da S_p)

Per ogni partizione scansioniamo S_p e per ogni chiave cerchiamo in countR :

Part.	S_p	Lookup	join_count
$p = 0$	[8]	$8 \in \text{countR}$, mult=1	+1
$p = 1$	[5, 5]	$5 \in \text{countR}$, mult=2 $\times 2$ record	+2 + 2 = +4
$p = 2$	[2]	$2 \in \text{countR}$, mult=2	+2
$p = 3$	[3, 7]	$3 \in \text{countR}$, mult=1; $7 \in \text{countR}$, mult=1	+1 + 1 = +2

Spiegazione della partizione 1

La chiave 5 appare 2 volte in R_1 e 2 volte in S_1 . Ogni record di S con chiave 5 contribuisce $\text{countR}[5] = 2$ match. Ci sono 2 record in S_1 con chiave 5, quindi il totale è $2 \times 2 = 4$ match. Questo è il meccanismo con cui l'algoritmo gestisce i **duplicati**.

1.5.7 Step 7 — Accumulazione

$$\text{join_count} = 1 + 4 + 2 + 2 = \boxed{9}$$

Verifica naive. Elenchiamo tutte le coppie (r, s) con $r.\text{key} = s.\text{key}$:

$$\begin{array}{lll} R[0]=5 \leftrightarrow S[1]=5 & R[3]=3 \leftrightarrow S[0]=3 & R[6]=7 \leftrightarrow S[5]=7 \\ R[0]=5 \leftrightarrow S[4]=5 & R[4]=2 \leftrightarrow S[2]=2 & \\ R[1]=2 \leftrightarrow S[2]=2 & R[5]=5 \leftrightarrow S[1]=5 & \\ R[2]=8 \leftrightarrow S[3]=8 & R[5]=5 \leftrightarrow S[4]=5 & \end{array}$$

Totale = 9 ✓ (corrisponde!)

1.6 Perché partizionare è vantaggioso

- Località di cache.** Senza partizionamento la hash table di R ha N_R entry $\Rightarrow$ enorme, non sta in cache; ogni lookup $\rightarrow$ cache miss. Con P partizioni, ogni hash table locale ha $\sim N_R/P$ entry; scegliendo P grande abbastanza, sta in L1/L2 $\Rightarrow$ ogni lookup è un cache hit.
- Indipendenza tra partizioni.** Le P partizioni non condividono dati $\Rightarrow$ nessun lock/sincronizzazione $\Rightarrow$ parallelismo naturale (*embarrassingly parallel*).
- Gestione della memoria.** Tante hash table piccole e di dimensione prevedibile invece di una gigante con possibili resize.

Il costo del partizionamento è $O(N)$ per relazione; il beneficio di cache supera di gran lunga questo costo per dataset grandi.

1.7 La gestione dei duplicati: perché “with Duplicates”

Se una chiave k appare m volte in R e n volte in S , produce $m \times n$ match:

- Build:** conta le occorrenze $\Rightarrow \text{countR}[\text{key}]++$
- Probe:** quando trova un match, aggiunge $\text{countR}[\text{key}]$ (non 1)

Esempio: $\text{key}=5$, $m = 2$ in R , $n = 2$ in $S \Rightarrow 4$ coppie matchanti, e l'algoritmo conta correttamente $2 + 2 = 4$.

1.8 Checksum: verifica senza materializzare le coppie

L'algoritmo non produce la lista delle coppie (sarebbe enorme). Verifica la correttezza con due checksum deterministici:

```
result.checksum1 += splitmix64(key) * multiplicity;
result.checksum2 += splitmix64(key ^ 0x9e3779b97f4a7c15ULL) *
    multiplicity;
```

- `splitmix64` è una funzione di mixing deterministica (stesso input → stesso output)
- Due checksum indipendenti (salt diversi) riducono la probabilità di collisioni accidentali
- Se la versione parallela produce gli stessi `join_count`, `checksum1`, `checksum2` ⇒ correttezza verificata

1.9 Mappa concettuale: Modulo 1 → Modulo 2

Modulo 1

Hai implementato e ottimizzato **una** funzione:

`compute_partition_id(key, P) → partition_id ∈ [0, P)`

Varianti: plain (scalare), autovec (GCC), AVX2 (intrinsics)

Benchmark: throughput in Mkeys/s, distribuzione tra partizioni

⇓ la funzione $h(key)$ viene usata qui

Modulo 2

Usi quella funzione **dentro** un algoritmo completo:

1. **Partizionamento**: chiama $h(key)$ per ogni record di R e S
 $\hookrightarrow$ Histogram + Scatter; chiamata $2 \times (N_R + N_S)$ volte
2. **Join locale**: non usa più $h(key)$; lavora su partizioni con Build+Probe
3. **Parallelizzazione**: scegli quali fasi parallelizzare con `std::thread`

Output: `join_count` + `checksum1` + `checksum2`

Report: strategie + grafici + discussione (max 5 pagine)

2 Teoria: Concetti Chiave dalle Lezioni

2.1 Metriche di Performance (Lezione 10)

$$\text{Speedup: } S(p) = \frac{T_{\text{seq}}}{T_{\text{par}}(p)} \quad \text{Efficienza: } E(p) = \frac{S(p)}{p} = \frac{T_{\text{seq}}}{T_{\text{par}}(p) \cdot p} \quad (1)$$

- Speedup ideale: $S(p) = p$ (lineare). In pratica sub-lineare per overhead.
- Efficienza ideale: $E(p) = 1$ (100%).
- **Scalabilità** (relative speedup): $\text{Scalab.}(p) = T_{\text{par}}(1)/T_{\text{par}}(p)$ — baseline = parallelo con 1 thread.

2.2 Strong vs Weak Scaling (Lezione 10)

Strong Scaling (problem size fisso, si variano i processori):

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}} \quad (\text{Legge di Amdahl}) \quad \Rightarrow \quad \lim_{p \rightarrow \infty} S(p) = \frac{1}{f}$$

Anche il 5% di codice seriale limita lo speedup massimo a $20\times$.

Weak Scaling (problem size cresce con p):

$$S(p) \leq f + p(1-f) \quad (\text{Legge di Gustafson}) \quad \text{WSE}(p) = \frac{T(PS(1), 1)}{T(PS(p), p)}$$

2.3 Work-Span Model (Lezione 15)

Un programma parallelo come DAG: T_1 = work totale, T_∞ = span (cammino critico).

$$\text{Lower bounds: } T_p \geq \max\left(\frac{T_1}{p}, T_\infty\right) \quad (2)$$

$$\text{Brent (upper): } T_p \leq \frac{T_1 - T_\infty}{p} + T_\infty \quad (3)$$

Parallelismo disponibile: T_1/T_∞ indica il massimo speedup teorico.

Applicazione al progetto

Ogni fase dell'algoritmo ha un proprio T_1 e T_∞ . Analizzare il rapporto T_1/T_∞ per ogni fase indica dove il parallelismo è conveniente.

2.4 Tipi di Parallelismo (Lezione 11)

Per questo progetto si usa **Data Parallelism**:

- Pattern **Map**: partizionamento delle relazioni (ogni thread gestisce un blocco di record)
- Pattern **Reduce**: accumulazione dei risultati parziali
- Il join locale per partizione è *embarrassingly parallel*

2.5 Data Partitioning e Workload Balancing (Lezione 14)

Strategia	Assegnamento	Quando usarla
Block	$\text{thread}_i \leftarrow \lfloor \frac{iN}{p}, \frac{(i+1)N}{p} \rfloor$	Workload regolare (histogram, scatter)
Cyclic	$t_i \rightarrow \text{thread}_{i \bmod p}$	Load balancing migliore
Block-Cyclic	chunk di dimensione c , ciclici	Compromesso
Dinamica	coda condivisa / work-stealing	Workload irregolare (join locale)

2.6 C++ Threads (Lezione 13)

```
1 #include <thread>
2 #include <vector>
3
4 std::vector<std::thread> threads;
5 for (int i = 0; i < nthreads; ++i) {
6     threads.emplace_back([i, &shared_data]() {
7         // lavoro del thread i
8     });
9 }
10 for (auto& t : threads) t.join(); // compilare con -pthread
```

Listing 2: Pattern base spawn/join

Regole chiave: ogni thread va joined/detached; `std::thread` è move-only; evitare oversubscription ($\text{threads} \approx \text{core}$); passare per `std::ref` con attenzione alla lifetime.

2.7 Cache e False Sharing (Lezione 5–6)

False Sharing: thread diversi scrivono su variabili nella stessa cache line (64B) $\Rightarrow$ invalidazione continua.

```
1 // MALE: contatori adiacenti
2 uint64_t counts[nthreads]; // false sharing!
3
4 // BENE: padding per separare le cache lines
5 struct alignas(64) PaddedCounter { uint64_t value = 0; };
6 PaddedCounter counts[nthreads];
```

Listing 3: Evitare false sharing con padding

3 Step 0: Setup dell'Ambiente

3.1 Struttura del progetto

```

module_2/
  Makefile
  README.md
  include/
    common.hpp          <- tipi, hash, utilita' dal Modulo 1
  src/
    hashjoin_seq.cpp    <- sequenziale (con la TUA hash)
    hashjoin_par.cpp    <- versione parallela
  results/              <- output dei benchmark
  report/
    report.pdf          <- report finale (max 5 pagine)

```

3.2 Makefile

```

1 CXX      = g++
2 CXXFLAGS = -O3 -std=c++20 -Wall -Wextra -pthread
3 INCLUDES = -Iinclude
4
5 all: hashjoin_seq hashjoin_par
6
7 hashjoin_seq: src/hashjoin_seq.cpp
8     $(CXX) $(CXXFLAGS) $(INCLUDES) $< -o $@
9
10 hashjoin_par: src/hashjoin_par.cpp
11     $(CXX) $(CXXFLAGS) $(INCLUDES) $< -o $@
12
13 clean:
14     rm -f hashjoin_seq hashjoin_par

```

Listing 4: Makefile

3.3 Compilazione e test iniziale

```

1 g++ -O3 -std=c++20 -Wall hashjoin_seq.cpp -o hashjoin_seq -pthread
2 ./hashjoin_seq -nr 5 -ns 8 -seed 13 -max-key 8 -p 4
3 ./hashjoin_seq -nr 1000000 -ns 2000000 -seed 42 -max-key 100000 -p 64

```

4 Step 1: Integrare la Funzione di Mapping del Modulo 1

Sostituisci `compute_partition_id` nel codice sequenziale:

```

1 // PRIMA (default del professore):
2 static inline uint32_t compute_partition_id(uint64_t key, uint32_t p) {
3     return (uint32_t)(key & (uint64_t)(p - 1U));
4 }
5
6 // DOPO (la tua dal Modulo 1):
7 static constexpr uint32_t HASH_A32 = 0x9E3779B9u;
8
9 static inline uint32_t compute_partition_id(uint64_t key, uint32_t p) {
10     unsigned shift = 32 - __builtin_ctz(p);
11     uint32_t k_lo = (uint32_t)(key);

```

```

12     uint32_t k_hi = (uint32_t)(key >> 32);
13     return ((k_lo ^ k_hi) * HASH_A32) >> shift;
14 }

```

Listing 5: Sostituzione della hash

Ottimizzazione

Il valore `shift` dipende solo da P ed è costante. Conviene pre-calcolarlo nel main e passarlo come parametro per evitare il costo di `__builtin_ctz` ad ogni chiamata.

La **stessa identica** funzione deve essere usata nella versione sequenziale e parallela.

5 Step 2: Capire il Codice Sequenziale

5.1 Generazione dei dati

R e S usano seed diverse (`seed` vs. `seed ^ 0xdead...`). Le chiavi sono generate con `splitmix64` e ridotte mod `max_key`. Più `max_key` è piccolo rispetto a N , più duplicati ci sono.

5.2 Histogram — $O(N)$

Scansione lineare, scrive su `hist[pid]` (array di P entry). **Parallelizzabile** con istogrammi locali + merge.

5.3 Prefix Sum — $O(P)$

Dipendenza loop-carried, ma P è piccolo. Tempo trascurabile $\Rightarrow$ **non parallelizzare**.

5.4 Scatter — $O(N)$

Scansione lineare con scritture **random** nell'output. Il cursore `next[pid]` è condiviso $\Rightarrow$ parallelizzare è il punto più delicato.

5.5 Join locale (Build + Probe)

Ogni partizione è **indipendente**. Il codice usa `std::unordered_map` (il professore nota che si può sostituire con strutture più efficienti).

6 Step 3: Analisi delle Fasi e Opportunità di Parallelismo

Citazione dalla traccia

“The goal is not simply to introduce parallelism, but to understand where parallelism is effective and where it is not.”

Fase	Compl.	Parall.?	Strategia	Note
Histogram	$O(N)$	✓ Sì	Istogrammi locali + merge	Facile, buon speedup
Prefix Sum	$O(P)$	× No	Sequenziale	P piccolo
Scatter	$O(N)$	~ Possibile	Offset pre-calcolati	Lock-free con local hist
Join locale	$O(N)/\text{part.}$	✓ Sì	1 thread per partizione(i)	Embarr. parallel
Accumulaz.	$O(P)$	× No	Sequenziale/reduce	P piccolo

Profila il sequenziale per individuare le fasi dominanti (tipicamente scatter + join locale per input grandi).

7 Step 4: Implementazione Parallela

7.1 Architettura generale

```

1 JoinResult partitioned_hash_join_parallel(
2     const std::vector<Record>& R, const std::vector<Record>& S,
3     uint32_t P, int nthreads)
4 {
5     PartitionedRelation Rpart = partition_relation_parallel(R, P,
6         nthreads);
7     PartitionedRelation Spart = partition_relation_parallel(S, P,
8         nthreads);
9     return parallel_join(Rpart, Spart, P, nthreads);
10 }

```

Listing 6: Struttura del programma parallelo

7.2 Histogram parallelo

Ogni thread calcola un istogramma locale sulla propria porzione (block distribution), poi si fa un merge sequenziale $O(P \times k)$.

```

1 // Istogrammi locali (evita false sharing!)
2 std::vector<std::vector<size_t>> local_hists(nthreads, std::vector<size_t>
3     >(P, 0));
4
5 std::vector<std::thread> threads;
6 for (int t = 0; t < nthreads; ++t) {
7     threads.emplace_back([&, t]() {
8         size_t start = (N * t) / nthreads;
9         size_t end = (N * (t + 1)) / nthreads;
10        for (size_t i = start; i < end; ++i)
11            ++local_hists[t][compute_partition_id(rel[i].key, P)];
12    });
13 }
14 for (auto& th : threads) th.join();
15
16 // Merge sequenziale
17 std::vector<size_t> global_hist(P, 0);
18 for (int t = 0; t < nthreads; ++t)
19     for (uint32_t pid = 0; pid < P; ++pid)
20         global_hist[pid] += local_hists[t][pid];

```

Listing 7: Histogram parallelo con istogrammi locali

7.3 Scatter parallelo lock-free (raccomandato)

Grazie agli istogrammi locali, ogni thread conosce *esattamente* dove scrivere:

```

1 // Calcolo offset per thread (sequenziale,  $O(P * nthreads)$ )
2 std::vector<std::vector<size_t>> thread_offset(nthreads, std::vector<
3     size_t>(P));
4 for (uint32_t pid = 0; pid < P; ++pid) {
5     size_t offset = global_begin[pid];
6     for (int t = 0; t < nthreads; ++t) {
7         thread_offset[t][pid] = offset;
8         offset += local_hists[t][pid];
9     }
10 }

```

```

10
11 // Scatter parallelo (ZERO lock! regioni di scrittura non overlap)
12 std::vector<Record> out(N);
13 for (int t = 0; t < nthreads; ++t) {
14     threads.emplace_back([&, t]() {
15         size_t start = (N * t) / nthreads;
16         size_t end   = (N * (t + 1)) / nthreads;
17         auto cursor = thread_offset[t]; // copia locale
18         for (size_t i = start; i < end; ++i) {
19             uint32_t pid = compute_partition_id(rel[i].key, P);
20             out[cursor[pid]++] = rel[i];
21         }
22     });
23 }

```

Listing 8: Offset per-thread-per-partizione + scatter senza lock

7.4 Join locale parallelo

Distribuzione ciclica delle partizioni tra i thread (load balancing migliore):

```

1 struct alignas(64) PaddedResult { JoinResult result{}; };
2 std::vector<PaddedResult> thread_results(nthreads);
3
4 for (int t = 0; t < nthreads; ++t) {
5     threads.emplace_back([&, t]() {
6         for (uint32_t pid = t; pid < P; pid += nthreads) {
7             JoinResult local = join_one_partition(Rpart, Spart, pid);
8             thread_results[t].result.join_count += local.join_count;
9             thread_results[t].result.checksum1  += local.checksum1;
10            thread_results[t].result.checksum2  += local.checksum2;
11        }
12    });
13 }
14 // ... join threads, poi reduce dei risultati

```

Listing 9: Join locale con distribuzione ciclica e padding anti false-sharing

7.5 Alternativa: distribuzione dinamica

```

1 std::atomic<uint32_t> next_partition{0};
2 // Ogni thread:
3 while (true) {
4     uint32_t pid = next_partition.fetch_add(1, std::memory_order_relaxed)
5     ;
6     if (pid >= P) break;
7     JoinResult local = join_one_partition(Rpart, Spart, pid);
8     // accumula...
9 }

```

Listing 10: Scheduler dinamico con atomic counter

7.6 Thread Pool vs Spawn-Join

Creare thread ad ogni fase introduce overhead. Opzione avanzata: creare i thread una volta e usare `std::barrier` (C++20) per sincronizzarli tra fasi, riducendo l'overhead $O(p)$ da Amdahl.

8 Step 5: Strategia di Validazione (Correctness)

1. **Input piccoli** ($N_R, N_S \leq 500$): confronto con `naive_join_verifier` ($O(N^2)$)
2. **Input grandi**: confronto `join_count`/checksum tra sequenziale e parallelo
3. **Multi-thread**: eseguire con $t = 1, 2, 4, 8, 16$ — tutti devono dare lo *stesso* risultato
4. **Edge cases**: poche partizioni, molti duplicati, thread > partizioni, partizioni vuote

```

1 for T in 1 2 4 8 16; do
2     echo "=== Threads=$T ==="
3     ./hashjoin_par -nr 1000000 -ns 2000000 -seed 42 \
4         -max-key 1000 -p 64 -t $T
5 done
6 # Tutti devono produrre gli STESSI join_count e checksum

```

Listing 11: Script di validazione

Opzionale ma consigliato: compilare con `-fsanitize=thread` per esporre data race.

9 Step 6: Performance Evaluation

Il report richiede **speedup**, **strong scaling** e **weak scaling**.

9.1 Strong Scaling

Fissare il problem size, variare i thread. Calcolare:

$$S(t) = \frac{T_{\text{seq}}}{T_{\text{par}}(t)}, \quad E(t) = \frac{S(t)}{t}$$

9.2 Weak Scaling

$N_R(t) = t \cdot N_R(1)$, $N_S(t) = t \cdot N_S(1)$. Calcolare:

$$\text{WSE}(t) = \frac{T(PS(1), 1)}{T(PS(t), t)}$$

9.3 Breakdown per fase

Fase	1 thread	8 thread	Speedup fase
Histogram R	xxx ms	xxx ms	x.xx
Scatter R	xxx ms	xxx ms	x.xx
Histogram S	xxx ms	xxx ms	x.xx
Scatter S	xxx ms	xxx ms	x.xx
Join locale	xxx ms	xxx ms	x.xx
Accumulazione	xxx ms	xxx ms	x.xx
TOTALE	xxx ms	xxx ms	x.xx

9.4 Regole di benchmarking

- Almeno 5 ripetizioni, prendere la mediana
- Warm-up iniziale
- Compilare con `-O3`
- Benchmark su un nodo dell'`spmcluster`
- Non misurare la generazione dei dati

10 Step 7: Scrivere il Report

Sezione	Contenuto	Pagine
1. Introduzione	Descrizione del problema e obiettivo	0.5
2. Strategia di Parallelizzazione	Analisi per fase, decisioni, come si evitano race/false sharing	1.5
3. Risultati Sperimentali	Speedup, weak scaling, breakdown per fase, piattaforma	2
4. Discussione	Bottleneck, confronto Amdahl, miglioramenti futuri	1

Grafici da includere: (1) Speedup vs #threads con linea ideale; (2) Efficienza vs #threads; (3) Breakdown per fase; (4) Weak scaling efficiency.

11 Checklist Finale

- ☐ Funzione hash del Modulo 1 integrata in `compute_partition_id`
- ☐ Versione sequenziale compila e produce output corretto
- ☐ Versione parallela implementata con `std::thread`
- ☐ Correctness: output parallelo = output sequenziale (per tutti i test)
- ☐ Edge cases testati
- ☐ Niente race condition (opzionale: testato con `-fsanitize=thread`)
- ☐ Niente false sharing: padding/allineamento dove necessario
- ☐ Benchmark strong/weak scaling eseguiti sul cluster
- ☐ Breakdown per fase misurato e riportato
- ☐ Makefile presente con istruzioni di compilazione
- ☐ README con istruzioni per compilare, eseguire, e validare
- ☐ Report PDF ≤ 5 pagine con grafici e discussione
- ☐ ZIP: `Modulo2_NomeCognome.zip` contenente tutto
- ☐ Email con oggetto “SPM Modulo2”

12 Appendice: Comandi Utili & Riferimenti

12.1 Compilazione

```

1 # Standard
2 g++ -O3 -std=c++20 -Wall -Wextra -pthread src/hashjoin_par.cpp -o
   hashjoin_par
3
4 # Thread sanitizer (debug race condition)
5 g++ -O1 -g -std=c++20 -fsanitize=thread -pthread src/hashjoin_par.cpp -o
   hashjoin_par_tsan
6
7 # Address sanitizer (debug memory)
8 g++ -O1 -g -std=c++20 -fsanitize=address -pthread src/hashjoin_par.cpp -o
   hashjoin_par_asan

```

12.2 Profilazione

```
1 # Perf (su Linux/cluster)
2 perf stat ./hashjoin_par -nr 10000000 -ns 20000000 -seed 42 -max-key
   1000000 -p 128 -t 8
3 perf record ./hashjoin_par ...
4 perf report
```

12.3 Riferimenti alle lezioni

Argomento	Lezione
Speedup, Efficienza, Amdahl, Gustafson	10-Metrics_and_Laws.pdf
Data/Stream/Task Parallelism	11-TypesOfParallelism.pdf
C++ auto, move, STL, lambdas	12-C++Essentials.pdf
std::thread, mutex, CV, future/promise	13-C++ConcurrencyBasics.pdf
Block/Cyclic distribution, Dynamic sched.	14-WorkloadBalancing.pdf
Work-Span, Brent's theorem, PRAM	15-ModelsOfComputation.pdf
Cache, False Sharing, Roofline model	15 & 6-Shared-Memory.pdf
SLURM	14-SLURM.pdf

Buon lavoro! Il professore valuta non solo la correttezza ma la capacità di ragionare sulle scelte di design e di supportarle con dati sperimentali.